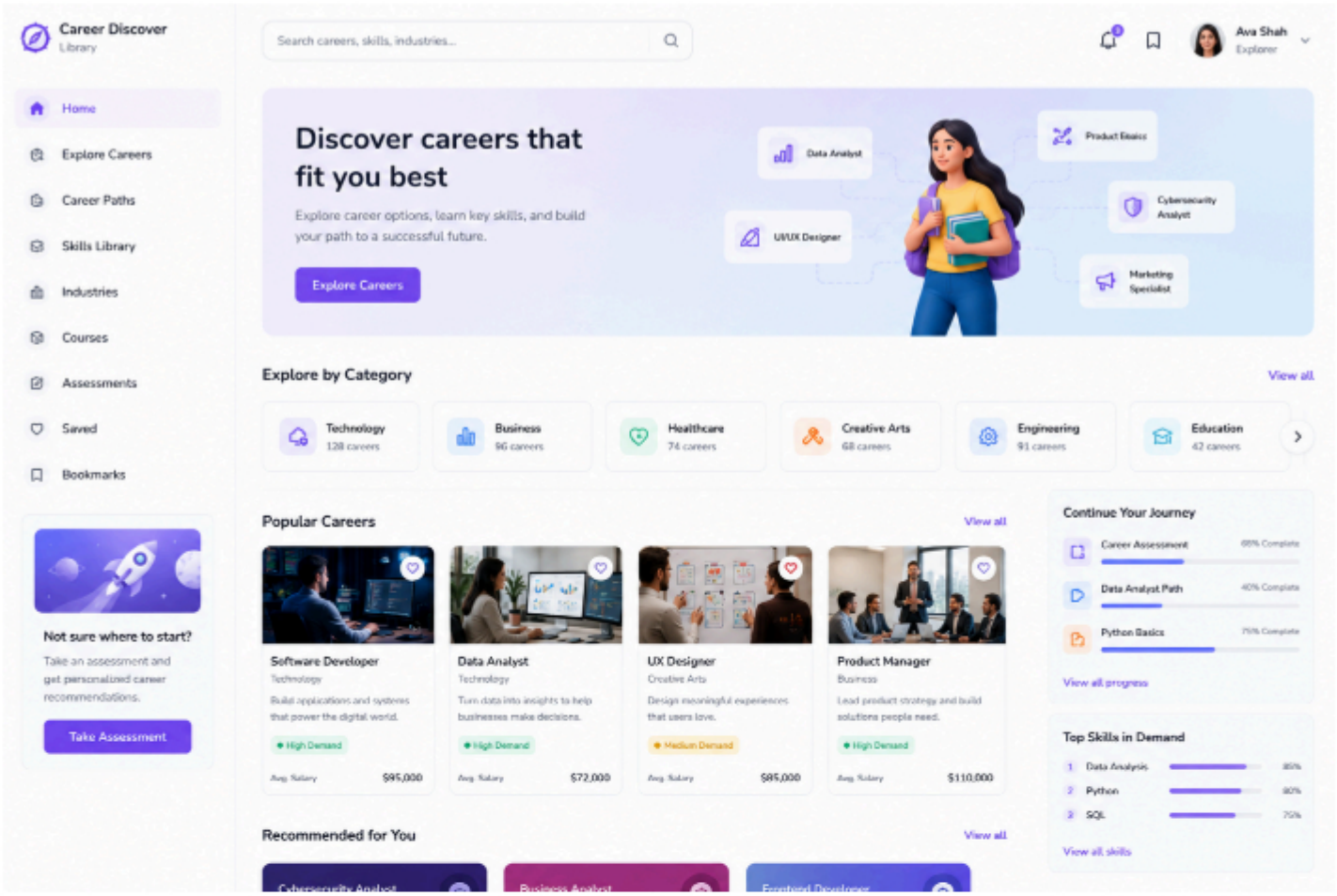

Fullstack Developer Practical Test

Project: Career Discovery Library

You are given **only one UI reference image**.

Your task is to transform that UI into a complete, production-like fullstack application.

You must independently:

- Infer product requirements
- Design user flows
- Design the database schema
- Build backend APIs
- Build frontend pages
- Integrate frontend + backend
- Deploy the application publicly on AWS

AI tools are allowed and expected.

Time Limit

Expected Completion Time

12 hours maximum (with AI assistance allowed).

The assignment is intentionally scoped to test:

- Speed of execution
- Product thinking
- AI-assisted engineering workflow
- Ability to ship a working fullstack product quickly

We care more about:

- Practical engineering decisions
- End-to-end completeness
- Working deployment
- Clean architecture

We care less about:

- Pixel-perfect UI
- Overengineering
- Huge feature sets

Final Expected Output

You must deliver **one live website URL** where we can:

- Register/login as a user
- Browse real dynamic career data
- Create, edit, and delete data
- Search and filter careers
- Save/bookmark careers
- Track learning progress
- Interact with persistent database-backed data
- Use the app like a real product

This must NOT be:

- A static landing page
- A frontend-only demo
- Hardcoded mock data

Product Concept

Career Discovery Library

A platform where users can:

- Discover careers
- Explore required skills
- Save careers
- Track learning progress
- Browse trending skills and categories

You should infer the detailed product flow from the UI image.

Suggested User Flows

1. Authentication

Users must be able to:

- Register
- Login
- Logout

---

2. Career Management

3. Search & Filters Careers

4. Skills Library

5. Saved Careers

6. Progress Tracking

7. Dashboard

Backend Requirements

Build a real backend API.

Suggested modules:

- Auth APIs
- Career APIs
- Skill APIs
- Category APIs
- Saved Career APIs
- Progress APIs
- Dashboard APIs

Backend expectations:

- Validation
- Authentication middleware
- Proper error handling
- Pagination support
- Search/filter query handling
- Environment variables
- Clean folder structure

Database Requirements

Use a real SQL database.

Suggested entities:

- users
- careers
- categories
- skills
- career_skills
- saved_careers
- user_skill_progress

Requirements:

- Foreign keys
- Proper relationships
- Many-to-many support
- Seed/sample data
- No hardcoded frontend data

Frontend Requirements

Frontend must connect to backend APIs.

Suggested pages:

- Login
- Register
- Dashboard
- Career listing
- Career detail
- Create/edit career
- Skills library
- Saved careers
- Progress tracking

Frontend expectations:

- Responsive design
- Reusable components
- Form validation
- Loading states
- Error states
- Empty states
- Delete confirmations

CRUD Requirements

At minimum, CRUD must work for:

- Careers
- Skills
- Categories

User-specific actions:

- Save/unsave careers
- Update progress

Deployment Requirement

Application must be publicly deployed on AWS.

Candidate must provide:

- Live frontend URL
- Live backend API URL
- Publicly accessible database-backed app
- GitHub repository
- README/setup instructions

The deployed application must work without local setup.

Recommended Free AWS Stack

Candidates may use AWS Free Tier:

Frontend

- Amazon S3 Static Hosting
- CloudFront (optional)

Backend

- Elastic Beanstalk
- EC2
- App Runner

Database

- Amazon RDS PostgreSQL/MySQL Free Tier

File Storage

- Amazon S3

Authentication

- JWT

Alternative Stack Allowed

To optimize for the 12-hour limit, candidates may also use:

Frontend

- Vercel
- Netlify

Backend

- Render
- Railway
- Fly.io

Database

- Neon
- Supabase
- PlanetScale

AWS deployment is preferred, but practical shipping matters more than infrastructure complexity.

Minimum Acceptance Criteria

Submission is acceptable only if:

- Live website works
- User registration/login works
- CRUD operations work
- Database persistence works
- Dynamic data is displayed
- Search/filter works
- Saved careers work
- Progress tracking works
- Backend APIs are functional
- Deployment is publicly accessible

Automatic Rejection Criteria

The submission will fail if:

- Frontend is static only

- No real backend exists
- No real database exists
- CRUD is incomplete
- Authentication is missing
- Deployment is broken
- Data is hardcoded
- App only works locally
- APIs are mocked only

Submission Requirements

Candidate must submit:

1. Live website URL
2. Backend API URL
3. GitHub repository
4. README
5. Database schema/migrations
6. Seed/sample data
7. Architecture explanation
8. Test user credentials

Evaluation Focus

We will evaluate:

Product Thinking

- Ability to infer requirements from UI

Backend Engineering

- API design
- Architecture
- Validation
- Security
- Database integration

Database Design

- Relationships